DP on Trees

INF494 – UFV – 2025/2

Dynamic Programming on Trees

Dynamic Programming (DP) is a technique to solve problems by breaking them down into overlapping subproblems which follow the optimal substructure. We all know of various problems using DP like subset sum, knapsack, coin change etc. We can also use DP on trees to solve some specific problems.

Collecting Coins

Given a tree T of N nodes, where each node i has Ci coins attached with it. You have to choose a subset of nodes such that no two adjacent nodes (i.e. nodes connected directly by an edge) are chosen and the sum of coins attached with nodes in chosen subset is maximum.

This problem is quite similar to 1-D array problem where we are given an array A1, A2, ..., AN; we can't choose adjacent elements and we have to maximise sum of chosen elements. For this problem we define:

- a state as ***dp(i)*** denoting answer for A1, A2, ..., Ai.
- the recurrence ***dp(i) = max{ Ai+dp(i-2), dp(i-1) }*** (two cases: choose Ai or not).

Collecting Coins

Now, unlike the array problem where the state is a solution for first *i* elements, in case of trees the state usually denotes a solution for a subtree.

For defining subtrees, we need to root the tree first. Say, if we root the tree at node 1 and define the state ***dp(v)*** as the answer for subtree of node *v*, then the final answer is ***dp(1)***.

Now, similar to array problem, we have to make a decision about including node *v* in the subset or not. If we include node *v*, we can't include any of its children (say v1, v2, ..., vn), but we can include any grandchild of *v*. If we don't include *v*, we can include any child of *v*.

Collecting Coins

So, we can write a recursion by defining maximum of two cases.
$dp(v) = max\{ \sum_{i=1..n} (dp(vi)), Cv + \sum_{i=1..n} (dp(j) \text{ for all children } j \text{ of } vi) \}$

As we see in most DP problems, multiple formulations can give us optimal answer. Here, from an implementation point of view, we can define an easier solution. We define two states, ***dp1(v)*** and ***dp2(v)***, denoting maximum coins possible by choosing nodes from subtree of node *v* if we include node *v* or not, respectively. The final answer is the maximum of two cases i.e. ***max{ dp1(1), dp2(1) }***.

Collecting Coins

Defining recursion is even easier in this case.

- $dp1(v) = Cv + \sum_{i=1..n} (dp2(vi))$ *(since we cannot include any of the children)*
- $dp2(v) = \sum_{i=1..n} max\{ dp1(vi), dp2(vi) \}$ *(since we can include children now, but we can choose not to)*

Implementation: you must notice that the answer for a node depends on answer of its children; you may write a recursive DFS, where you first call recursively for all children and then calculate answer for current node. ⇒ O(n)

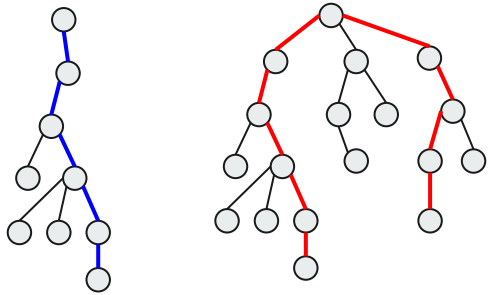
Diameter of a tree

Given a tree T of N nodes, calculate longest path between any two nodes(also known as diameter of tree).

First, let's root tree at node 1. Now, we need to observe that there would exist a node *x* such that:

- longest path starts from node *x* and goes into its subtree (blue line, let's call it *f(x)*).
- longest path starts in a subtree of *x*, passes through *x* and ends in a subtree of *x* (red line, *g(x)*).

We can get the diameter by taking the maximum of *f(x)*, *g(x)* for all nodes *x*.



Diameter of a tree

We need to define how to calculate the maximum path length in both cases.

Let's say a node v has n children v1, v2, ..., vn.

- *f(v) = 1 + max{ f(v1), f(v2), …, f(vn) }*
- *g(v) = 2 + sum of two max values in set { f(v1), f(v2), …, f(vn) }*

Note: we can recursively define *f(v)*, because we are looking at maximum path length possible from children of *v*, then we take the maximum one. So, optimal substructure is being followed here. This is quite similar to DP except that now we are defining functions for nodes and defining recursion based on values of children. This is what DP on trees is.

Diameter of a tree

There is an easiest way to find the diameter of a tree…

- perform a BFS or DFS from any arbitrary node to find the farthest node from it.
- run a second BFS or DFS starting from that farthest node to find the new farthest node. The distance between these two nodes is the diameter of the tree.

Tree matching

Find the maximum matching of a tree (the largest set of edges such that no two edges share an endpoint).

Root the tree at node 1, allowing us to define the subtree of each node.

- Let ***dp2[v]*** represent the maximum matching of the subtree of *v* such that we don't take any edges leading to some child of *v*.
- Let ***dp1[v]*** represent the maximum matching of the subtree of *v* such that we take one edge leading into a child of *v*. (note that we can't take more than one edge leading to a child, because then two edges would share an endpoint)

Tree matching

Taking No Edges:

Since we will take no edges to a child of *v*, the children of *v* can all take an edge to some child or not. Additionally, observe that the children of *v* taking an edge to a child will not prevent other children of *v* from doing the same. In other words, all of the children are independent.

- $dp2[v] = \sum_{u \text{ in child}(v)} max\{ dp1[u], dp2[u] \}$

Tree matching

Taking One Edge:

The case where we take one child edge of *v* is a bit trickier. Let's assume the edge we take is *(v, u)*

- we take the edge *(v, u)*, we add ***dp2[u]+1***
- we can't take any children of *u*, then for the other children, we add $\sum_{w \text{ in child}(v),\ w \neq u} max\{dp1[w], dp2[w]\}$

Fortunately, since we've calculated *dp2[v]* already, this expression simplifies to:

$$dp2[v] - max\{ dp2[u], dp1[u] \}$$

Overall, to calculate the transitions for *dp1[v]* over all possible children *u*:

$$dp1[v] = max_{u \text{ in child}(v)} \{ dp2[u] + 1 + dp2[v] - max\{ dp2[u], dp1[u] \}$$

Tree matching

There is an easiest way to solve the tree matching problem…

- keep matching a leaf with the only vertex adjacent to it while possible (greedy)

Counting sub trees

Given a tree T and an integer K, find number of different sub trees of size less than or equal to K.

"sub tree" is a subset of nodes of original tree such that this subset is connected (≠ subtree)

Note: always think by rooting the tree! So, say that tree is rooted at node 1.
Let *S(v)* be the subtree rooted at node *v* (standard subtree definition, *S(v)* includes all nodes in subtree of *v*)

Counting sub trees

Let's try to count total number of sub trees of a tree first. Then, try to use same logic for original problem.

Let $f(v)$ be the number of sub trees of $S(v)$ which include node v (i.e., v is the root of the sub trees).

For each child u of node v, we have two options: whether to include them in sub tree or not.
If u is included, then there are $f(u)$ ways, otherwise there is only one way (since we can't choose any nodes from $s(u)$, otherwise the sub tree we are forming will get disconnected).

$f(v) = \prod_{i=1..n} (1 + f(vi))$

Is our solution complete? Can we just return f(1), which counts number of sub trees of T rooted at 1?

Counting sub trees

Now, back to our original problem. We are trying to count sub trees of T whose size doesn't exceed K.

We need one more state in the DP at each node. Let $f(v, k)$ be the number of sub trees with k nodes and v as root. Now, we can define recurrence relation for this, using its direct children nodes v1, v2, ..., vn.

To form a subtree with $k + 1$ nodes rooted at v, let's say $S(vi)$ contributes ai nodes. As node v is included in the $k+1$, we must have $k = \sum_{i=1..n} ai$. Thus, $f(v, k)$ is sum of $\prod_{i=1..n} f(vi,ai)$ for all possible distinct sequences a1, a2, ..., an.

To compute this at node v, we will form another DP: let $dp(i, j)$ be the number of ways to choose j nodes from subtrees defined by v1, v2, ..., vi. The recurrence can be defined as $dp1(i, j) = \sum_{k=1..n} dp(i-1, j-k) f(vi,k)$

Finally, $f(v,k) = dp(n,k)$ and the solution is $\sum_{i=1..k} f(v,i)$ for all nodes v in T $\Rightarrow O(nk^2)$

T1 structurally similar to T2

Given two rooted trees T1 and T2, make T1 structurally similar to T2. For doing that you can insert leaves one by one in any of the trees. You have to tell the minimum number of insertions required to do so.

Let's say both trees are rooted at nodes 1. Now, T11 has children u1,u2,...,un and T21 children v1,v2,...,vm We are going to create a mapping between nodes in set *u* and *v*, i.e. we are going to make subtree of some node *ui* exactly same as *vj*, for some *i*, *j*, by adding required nodes. If $n \neq m$, then we are going to add the whole subtree required.

How do we decide which node in T1 is mapped to which in T2? Again, we use DP here. Let *dp(i,j)* be the minimum additions required to make subtree of node *i* in T1 similar to subtree of node *j* in T2.

T1 structurally similar to T2

Let's say node *i* has children u1, u2, ..., un and node *j* has children v1, v2, ..., vm.

If we assign node *ui* with node *vj*, then the cost is going to be *dp(vi, vj)*. Now, to all nodes in *u*, we have to assign nodes from *v* such that total cost is minimised. This can be solved by solving assignment problem!

In assignment problem there is a cost matrix C, where *C(i, j)* denotes cost if task *i* is assigned to person *j*. Our aim is to assign one task to one person such that total cost is minimised. This can be done in $O(N^3)$, if there are N tasks. Here in our problem *C(i, j)=dp(vi, vj)*, and by solving it, we can get value of *dp(i, j)*.

Tutorial / source / more info

The contents of these slides can be found here:

- USACO Guide: DP on Trees – Introduction: https://usaco.guide/gold/dp-trees?lang=cpp
- Codeforces: DP on Trees – Tutorial: https://codeforces.com/blog/entry/20935

More info here:

- IOI Training: DP on Trees and DAGs: https://noi.ph/training/weekly/week5.pdf
- USACO Guide: DP on Trees – Solving for all roots: https://usaco.guide/gold/all-roots?lang=cpp